Palantir × Spaceport_SK — Research & Pitch Plan

> Working document for the application to the 5th cohort of the **Spaceport_SK** incubation programme (2026), operated by the Slovak Space Office (SARIO) in cooperation with the Slovak Ministry of Education, Research, Development and Youth.

---

1. Executive Summary

Palantir is an open-source **Ground Segment as a Service (GSaaS) Digital Twin** — a bridge between astrodynamic simulation (Orekit/SGP4) and operational mission control (Yamcs) over the standard CCSDS 133.0-B-1 Space Packet Protocol. At the time of application, I have a **working prototype with a bidirectional TM/TC link, a Docker Compose stack, and a fully operational XTCE decoding pipeline** — exactly what Spaceport_SK expects from a participant ("a good idea suffices, but a prototype is a plus").

For the 6-month programme, I have prepared a **pedagogically structured backlog** (`FEATURE-v2-ext.md`) of 13 tickets (PAL-101 through PAL-504) covering HMI, flight dynamics, multi-payload XTCE, Yamcs Java plugins, HPC conjunction assessment, and ML-based anomaly detection — demonstrating the ability to scale the project through student team integration (candidates from Slovak technical universities, which the programme explicitly supports).

Goal of participation: product validation, connection to the Slovak space ecosystem, preparation for **ESA BIC Slovakia** (the programme explicitly positions itself as the validation path before BIC entry), access to the international Advisory Board (ESA, EUSPA, YESS, Satlantis), and prize money at Demo Day (29 September 2026).

---

2. Spaceport_SK — What We Need to Know for the Application

Programme at a Glance

Attribute	Value
Operator	Slovak Space Office @ SARIO + Ministry of Education (MŠVVaM SR)
Format	Hybrid incubator (mentoring + online workshops)
Duration	April – September 2026 (6 months, 3 phases)

Target audience	Early-stage startups, **student teams** (technical/scientific/business disciplines), individuals with an idea or prototype
Output	Demo Day pitches before an expert jury, prize money, pipeline into ESA BIC

Key Dates

Date	Milestone
20 April 2026	**Application deadline** (spaceoffice@sario.sk)
28 April 2026	Kick-off
April – May	Phase 1: Validation (Business Canvas, technology validation, go-to-market)
June	Phase 2: Investment Readiness (pitch deck, VC Q&A, ESA BIC overview)
September	Phase 3: Finalisation + Final Advisory Board review
29 September 2026	**Demo Day** (competition with prizes before expert jury)

Application Requirements

1. **Project theme**
2. **Team composition**
3. **Short project description**

What the Programme Provides

- Mentoring from experienced space sector professionals
- Access to the Slovak space ecosystem (companies, research institutions, investors)
- International feedback via the Advisory Board (ESA, EUSPA, YESS, Satlantis)
- **Preparation for ESA BIC Slovakia** (official "validation before entry")
- Prize money for Demo Day winners

Focus Areas (Where Palantir Fits)

The programme explicitly lists the following priorities — items in **bold** are direct fits for Palantir:

- **Satellite operations and remote sensing ← direct fit**
- Earth observation and environmental monitoring
- AI and machine vision applications ← *adjacent (roadmap)*
- **Satellite data utilisation and downstream services ← direct fit (GSaaS)**
- Heat shield technology
- **Space debris monitoring and mitigation ← adjacent (PAL-501/502 collision avoidance)**
- Propulsion systems for small satellites

- **Autonomous systems ← adjacent (automated COLA loop)**
- Geoinformation solutions

Previous Winners — What the Jury Rewards

Year	Winner	Theme
2025 (R4)	**Aerostacks**	Avalanche risk prediction via remote sensing + AI
2024 (R3)	**Datafrost Space**	Marketplace connecting satellite data users and providers
2023 (R2)	**Straton**	Autonomous glider for stratospheric probe recovery
2022 (R1)	**SpaceScavengers**	Multi-agent systems for space debris management

Pattern: the jury rewards solutions that (a) have a clear commercial use case, (b) leverage a modern stack (AI, data platforms), (c) solve a real problem visible from or toward space. **Palantir fits elegantly into this trajectory as "an open-source ground segment for NewSpace mega-constellations."**

---

3. Why Palantir Belongs in Spaceport_SK

Strategic Fit

1. **It has a working prototype, not just a slide deck.** The programme does not require this — but it is a strong differentiator. A live demo with real telemetry streaming in 60 seconds is more persuasive than any pitch.
2. **It addresses a real commercial gap.** Legacy ground segment systems (SCOS-2000, EPOCH) are proprietary and expensive; Yamcs on its own is just the core — Palantir adds the physics-driven digital twin layer and a GSaaS-ready deployment model.
3. **Mega-constellations = data deluge problem.** Java 21 provides two independent scaling axes for Palantir: Virtual Threads for thousands of concurrent blocking I/O operations (per-satellite UDP downlinks, per-satellite TC listeners) and multi-core platform-thread CPU parallelism for SGP4/SDP4 propagation (which is CPU-bound). Combined, both support projected scaling towards ~10,000 satellites on a single 16-core COTS server — subject to empirical benchmarking as part of the Feature B roadmap. This is directly **Killer Feature B** from `FEATURES_v2.md`.
4. **The ESA BIC path is explicit.** The programme positions itself as "validation before ESA BIC." Palantir is exactly the profile of project BIC looks for (technical foundation + commercialisation potential in GSaaS).
5. **Student-ready scale-up plan.** The 13-ticket backlog in `FEATURE-v2-ext.md` demonstrates that the project can absorb student/junior contributors and generate deployable artefacts — precisely the kind of scaling Spaceport_SK values.

Differentiation from Previous Winners

- Datafrost Space solved a **marketplace** for satellite data → Palantir solves the **infrastructure** that processes and distributes that data into mission control.
- SpaceScavengers solved **debris management** at the agent level → Palantir provides the **ground segment** on which such agents operate, including an XTCE command definition for `FIRE_THRUSTER` (PAL-502).
- Palantir does not compete — it **complements** existing winning projects as a ground infrastructure layer.

---

4. Current Prototype (Demo-Ready State)

Everything runs in a single `docker compose up --build`. That is **the critical point for the pitch** — mentors and jurors must be able to run the demo locally within 5 minutes.

Implemented (Verified on `master` Branch)

Physics & Orbital Mechanics

- **Orekit 12.2 SGP4/SDP4 propagator** with runtime loading of `orekit-data.zip` (leap seconds, Earth orientation parameters, planetary ephemerides).
- **Coordinate transformation pipeline** TEME → ITRF → WGS84 geodetic via `OneAxisEllipsoid` and IERS-2010 conventions.
- **Hot-swap TLE via REST API** (`POST /api/orbit/tle`) using `AtomicReference<TLEPropagator>` — zero-downtime propagator replacement.
- Default ISS TLE loads on startup → telemetry flows immediately after boot, no manual steps required.

CCSDS Downlink (TM)

- **Full CCSDS 133.0-B-1 Space Packet implementation** at the `ByteBuffer` level (6 B primary header + 12 B payload).
- APID 100, 14-bit sequence counter via `AtomicInteger`, standalone grouping flags.
- **IEEE 754 big-endian float encoding** for lat/lon/alt (3 × 4 B).
- **UDP transport** on port 10000 (simulates S-Band downlink).
- **DEBUG-level hex dump logging** of transmitted packets (recent commit `c5729b1`).

Yamcs Mission Control (Docker)

- Custom Yamcs 5.12.2 image (`yamcs/example-simulation` base) with instance `palantir`.
- `UdpTmDataLink` on port 10000 with `GenericPacketPreprocessor` (local generation time, no CRC).

- **XTCE Mission Database** (`mdb/palantir.xml`):
  - Abstract `CCSDS_Packet_Base` container (6 B header).
  - `Palantir_Nav_Packet` (APID=100 restriction, 3 × IEEE 754 floats).
  - `CommandMetaData` with PING (0x01) and REBOOT_OBC (0x02) as uint8 opcodes.
- **Realtime processor** with `StreamTmPacketProvider` + `StreamTcCommandReleaser` + `StreamParameterProvider`.
- **Archive services** (`XtceTmRecorder` , `ParameterRecorder` , `CommandHistoryRecorder`) with persistent Docker volume (`palantir_yamcs_data`).
- Web UI on port 8090, CORS enabled.

Telecommand Uplink (TC) — Bidirectional Loop

- **`UdpTcDataLink`** in Yamcs sends TC packets to `palantir-core:10001` via Docker DNS.
- **`UdpCommandReceiver`** in Spring Boot listens on UDP 10001 in a Java 21 Virtual Thread executor.
- **Opcode dispatch:** 0x01=PING, 0x02=REBOOT_OBC, 0x03=SET_TRANSMIT_POWER (the last one in code only, not yet in the XTCE MDB).
- An operator clicks a button in the Yamcs web UI → the packet reaches Spring Boot logs via 2 hops in under 10 ms.
- **Design note — asymmetric CCSDS framing:** the TM downlink is fully CCSDS 133.0-B-1 compliant (Primary Header, APID 100, IEEE 754 big-endian payload). The TC uplink is deliberately a minimalist 1-byte opcode protocol over UDP — the XTCE `CommandContainer` defines only a uint8 `OpCode` argument with no CCSDS Primary Header inheritance, so Yamcs `StreamTcCommandReleaser` emits exactly 1 byte and Spring Boot reads it as `data[0]`. Full CCSDS TC Space Packet framing (Primary Header + APID 101 + Seq Flags + a dedicated `CcsdsTelecommandParser` in Spring Boot) is an explicit Phase 3b roadmap item — not a bug, but a deliberate PoC scope decision to accelerate the bidirectional loop.

Engineering Quality

- **Java 21** with `spring.threads.virtual.enabled=true` (Project Loom).
- **Multi-stage Dockerfile** (Maven + JDK build → JRE runtime).
- **`depends_on: service_started`** (not `service_healthy`) — a deliberate fix for the DNS race condition between Yamcs `UdpTcDataLink` init and the `palantir-core` joining the Docker network.
- **JUnit 5 + Mockito test suite** (6 tests: `@WebMvcTest` for the controller, `@SpringBootTest` for the propagation service, plus an integration test for full context load).
- **JaCoCo coverage reporting** (`target/site/jacoco/index.html`).
- **Complete documentation**: README.md, ARCHITECTURE.md, FEATURES.md, FLOW.md (byte-by-byte CCSDS encoding walkthrough), FEATURES_v2.md (killer features), FEATURE-v2-ext.md (student backlog).
- **Javadoc across all main classes** (recent commit `4f9d189`).

What This Means for the Pitch

The prototype is a **reference implementation**, not an academic script. It compiles, runs, has a baseline test suite, is documented, and uses a real-world protocol (CCSDS 133.0-B-1) and a real-world open-source mission control system (Yamcs — used by selected ESA small-sat programmes such as SmallGEO, and by several commercial NewSpace operators). That is exactly what the jury needs to see in the first 60 seconds.

---

5. Roadmap & Plans (What the Programme Should Fund/Mentor)

Phase 3b — Advanced Telecommanding (already in `FEATURES.md`)

- **CCSDS Telecommand Parser** with full header validation (APID, Sequence Flags).
- `CommandExecutorService` — decoupling opcode dispatch from the UDP receiver.
- **XTCE command expansion** (SET_TRANSMIT_POWER, new operational commands).
- **Physical reaction to TCs:**
  - `THRUST_MANEUVER` → dynamically modifies the propagator (delta-V applied to `SpacecraftState`).
  - `CHAOS_MONKEY` fault injection → simulated subsystem failure (e.g. battery voltage → 0 V).

Killer Features from `FEATURES_v2.md`

1. **Feature A — Predictive Orbital Shadowing (Model Residual Fault Detection).** The Orekit engine runs in parallel with live telemetry — an ideal orbit state is predicted continuously, Yamcs compares Δ between simulated and received position, and a `ParameterAlarm` triggers when the threshold is exceeded (typically excessive drag residual, an unexpected manoeuvre, or catastrophic orbit decay). This is an application of the standard digital-twin residual-based fault detection pattern to the orbit subsystem — the first open-source GSaaS to wire this directly into the Yamcs `ParameterAlarm` pipeline. Scope is orbit dynamics, not attitude/power/thermal (those are covered in the Epic 3-4 backlog).
2. **Feature B — Multi-Axis Scaling.** Java 21 enables combining two orthogonal scaling strategies: Virtual Threads for thousands of concurrent per-satellite blocking I/O loops (UDP sockets, REST listeners) and multi-core platform-thread parallelism for CPU-bound SGP4/SDP4 propagation. Target: projected scaling towards ~10,000 satellites on a single 16-core COTS server, verified by empirical benchmarking. Directly addresses the mega-constellation data deluge without requiring horizontal scale-out in the first phase.

3. **Feature C — Closed-Loop Command Verification.** A bidirectional ACK handshake: a command is not marked `COMPLETED` in the MDB until a confirming telemetry packet arrives from the OBC, validated through an XTCE `ManualVerifier` + `TransmissionConstraint` gate. Eliminates "blind commanding" — the mandatory pattern for ESA institutional operations and for safety-critical COLA procedures. A follow-up in the long-term backlog is CCSDS SDLS HMAC-SHA256 authentication of TC packets for full cyber-resilience.

Student Scale-Up (from `FEATURE-v2-ext.md`)

6-month programme, 2 students, 13 tickets, producing a portfolio of deployable artefacts:

Epic	Tickets	Domain
1 — HMI	PAL-101, PAL-102	CesiumJS 3D globe, WebSocket telemetry, TC control panel
2 — Flight Dynamics	PAL-201, PAL-202	Python pandas/matplotlib pipeline, AOS/LOS pass predictions
3 — Multi-Payload XTCE	PAL-301, PAL-302	APID 200 extension pattern, multi-stream archive validation
4 — Yamcs Plugins	PAL-401, PAL-402, PAL-403	Custom Java algorithms (quaternion→Euler), payload simulator microservice, Testcontainers stress testing
5 — HPC + AI	PAL-501, PAL-502, PAL-503, PAL-504	Orekit Monte Carlo conjunction assessment, automated collision avoidance TC loop, synthetic anomaly dataset, PyTorch autoencoder + ONNX export

Why this matters to Spaceport_SK: the programme explicitly targets **student teams** — and I have a ready-to-execute backlog into which I can immediately onboard 2 students. The project was presented in early April 2026 at **Univerzita Mateja Bela (UMB) in Banská Bystrica**, and I am currently awaiting the students' response on whether they will commit to the integration work. This is a unique scaling plan that other participants are unlikely to match.

Long-Term (Backlog)

- Ground station visibility (AOS/LOS for ESTEC, Kiruna, or a hypothetical Slovak GS).
- CCSDS SDLS — HMAC authentication of TC packets (security for institutional operation).
- Kubernetes Helm chart for horizontal scaling of the telemetry processor.
- Cloud-native deployment on AWS Ground Station or Azure Space.

Concrete Demo Day 2026 Target: Automated Collision Avoidance Bundle

> *Added 2026-04-11 after a scope discussion about solo vs. team execution.*

Target deliverable: a complete, end-to-end Automated Collision Avoidance (COLA) flow running locally on my own PC via `docker compose up`, ready for a live demo at Demo Day (29 September 2026). This is the moment meant to stick with the jury — and it also directly addresses the programme focus area "*space debris monitoring and mitigation*".

What's in the bundle (three pieces bundled as ONE demo package, not three independent features):

1. **PAL-501 — HPC Conjunction Assessment & Monte Carlo Pc** (backlog estimate 100 h) Standalone Maven project using Orekit 12.2, ingests the CelesTrak GP catalog (~500 objects in local validation mode), screens minimum distance over a 7-day propagation window, performs Monte Carlo Pc estimation, outputs a JSON report plus the top-10 conjunctions posted to the Yamcs event log via REST API.
2. **PAL-502 — Automated Collision Avoidance Telecommanding** (backlog estimate 80 h) `FIRE_THRUSTER` XTCE MetaCommand with `TransmissionConstraint` + `ManualVerifier` (command waits for operator APPROVE); Python script polls Yamcs events, detects Pc > 10^{-4}, and queues a thruster command with computed delta-V. Safety gate: the operator must manually approve via the Yamcs UI — the command never releases without it.
3. **Phase 3b `THRUST_MANEUVER` physics reaction** (estimated ~15–25 h on top of the backlog) *This piece is explicitly missing from the `FEATURE-v2-ext.md` backlog but is critical for the demo narrative.* In `OrbitPropagationService`, we need to react to opcode 0x04 (FIRE_THRUSTER): apply the delta-V to the current `SpacecraftState`, build a new `TLEPropagator`, and atomically swap it via the existing `AtomicReference<TLEPropagator>` pattern. Result: in the Yamcs UI, one can see the orbit **visibly changing in real time** after clicking APPROVE.

Why the third piece is non-negotiable: PAL-501 + PAL-502 as originally written stop at "command reaches `SENT` status" — i.e. Spring Boot receives the packet, logs the opcode, and nothing else happens. That is a technically correct closed loop, but visually it is just a line in the log. Adding the physics reaction transforms the demo from "impressive plumbing" into **"the operator just saved the spacecraft from a debris collision"** — that is the emotional moment the Demo Day jury will remember and associate with the project's name.

Effort budget:

Component	Backlog estimate	Solo realistic	Note
PAL-501	100 h	60–80 h	Backlog assumes a student; senior solo is faster
PAL-502	80 h	40–60 h	Backlog assumes a student
Phase 3b physics wiring	—	15–25 h	Not in the backlog; critical for the demo narrative
Polish + integration + rehearsal	—	20–30 h	Demo video, documentation, dry runs

Total (solo)	**180 h**	**135–195 h**	

Timeline reality check (2026-04-11 → 2026-09-29 ≈ 24 weeks):

Weekly budget	Assessment
> 15 h/week solo	Safe margin; handles full rigor (N=10,000 Monte Carlo, full test coverage)
~10 h/week solo	On the edge; rigor must be descoped (see below)
5–8 h/week solo	**UMB students are not nice-to-have, they are a blocker**; without them the demo does not make the date
With 2 UMB students pair-programming PAL-501	Comfortable timeline; I focus on PAL-502 + Phase 3b wiring + polish

Team status (as of 2026-04-11): the project was presented at UMB in Banská Bystrica in early April 2026. Awaiting the students' response on potential involvement. **The demo target holds regardless of the outcome** — if UMB falls through, execution continues solo with aggressive descoping.

Descope strategy for the solo scenario (if UMB falls through):

Guiding principle — **descope rigor, not features**. The demo narrative must remain intact.

- Monte Carlo `N = 1,000` samples instead of `10,000` (saves ~15–20 h of compute + debug time; Pc accuracy is sufficient for a demo, not for a paper)
- `--catalog-limit=500` permanent, not a CLI flag. Skip HPC/Slurm deployment entirely (saves ~25–30 h of infra work)
- 80% test coverage as an aspiration, not an acceptance gate (saves ~10 h)
- Skip the PAL-502 HMI integration (PAL-102 extension) — use the built-in Yamcs commanding UI for APPROVE/REJECT buttons (saves ~5 h)
- **Do NOT skip under any circumstances:** the Phase 3b physics reaction. It is non-negotiable for the demo narrative — without it, the entire emotional payoff collapses.

Risk-ordered sprint plan (solo scenario):

Sprint	Window	Goals
1	April – May	PAL-501 Maven scaffold, CelesTrak GP CSV parser, `--catalog-limit=10` first run validates data flow
2	May – June	PAL-501 screening pass (TLE → TLE distance over 7-day window), Monte Carlo (N=1,000), local `--catalog-limit=500` completes in < 10 min
3	June – July	PAL-501 Yamcs event posting via REST, `conjunction_report.json` complete, PAL-502 XTCE `FIRE_THRUSTER` + `ManualVerifier`

4	July – August	PAL-502 Python COLA script, E2E test with a synthetic conjunction event, operator approval flow through the Yamcs UI
5	August	**Phase 3b** `THRUST_MANEUVER` **wiring** in `OrbitPropagationService`, delta-V applied to `SpacecraftState`, atomic propagator swap, verification that the orbit visibly changes in the Yamcs UI after APPROVE
6	September	Polish, integration testing, backup 60 s demo video, Demo Day rehearsal, final Advisory Board feedback

Critical gate after Sprint 5: before the start of September, the end-to-end demo must work from a clean `docker compose up` on `master`. No "only runs on my machine" shortcuts, no manual steps beyond `docker compose up` + curl. If the gate fails, all of September is the buffer for fixing and rehearsing.

---

6. Commercialisation Potential (for Phases 1 and 2)

Problem Statement

Legacy ground segment systems (SCOS-2000, EPOCH, open-source OpenC3) either cost hundreds of thousands of euros per seat or fail to address mega-constellations (10,000+ sats) and predictive maintenance. NewSpace operators need a low-cost, cloud-native, scalable GSaaS — and academic solutions do not deliver it.

Target Customers

1. **NewSpace operators of small-sat constellations** — need a fast, scalable ground segment without the CapEx of SCOS-2000.
2. **ESA BIC incubated companies** — need a reference platform for their payload projects (e.g. SatelliteVu, ICEYE scale-ups).
3. **National space agencies of smaller countries** — Slovakia, Czechia, Hungary, Croatia: need a ground segment but cannot afford ESA-tier systems.
4. **Defence primes / SSA operators** — COLA loop, conjunction assessment, and fault injection are directly defence-relevant.

Revenue Model Scenarios

- **Open-core SaaS:** core is open-source, plugins and enterprise support are commercial (Red Hat / Grafana Labs model).
- **Per-satellite GSaaS licensing** ($/sat/month) via cloud hosting.
- **Consulting + integration** for institutional customers (SARIO / Ministry of Defence / national agencies).

- **ESA BIC startup runway** — ~€50k direct financial support (grant or zero-interest loan depending on the specific BIC) + significant in-kind services (technical support, facility access, mentoring, IPR advice). Exact terms and figures for ESA BIC Slovakia TBD in Phase 2 of the programme — do not conflate direct financing with in-kind valuation in investor pitches.

Differentiators vs. the Competition

Capability	Palantir	SCOS-2000	OpenC3	AWS Ground Station
Open source	✅	❌	✅	❌
Cloud-native	✅	❌	⚠️	✅
Physics digital twin	✅	❌	❌	❌
CCSDS 133.0-B-1	✅	✅	✅	✅
XTCE MDB	✅	✅	❌	⚠️
Virtual Thread scaling	✅	❌	❌	❌
ESA ground segment lineage (Yamcs)	✅	✅	❌	❌

(This table is rough — during the preparatory phase, exact competitor capabilities need to be verified.)

---

7. Short Presentation Structure (Demo Day + Advisory Board Feedback)

The programme has two presentation opportunities:

1. **Kick-off / Advisory Board feedback** (April-May) — 3–5 minute pitch, technically detailed.
2. **Demo Day** (29 September 2026) — typically 5–7 minutes for the jury, pitch + live demo.

Proposed 7-Slide Pitch Deck (Adaptable to Both)

#	Slide	Content	Time
1	**Hook — Problem**	"NewSpace operators need ground segments for 10,000+ satellites. Legacy systems cost millions. Open-source alternatives do not exist." Show one growth chart of mega-constellations (Starlink, Kuiper, OneWeb, Guowang).	30s
2	**Solution**	"Palantir = open-source GSaaS Digital Twin. Orekit physics + Yamcs mission control + CCSDS 133.0-B-1 over Virtual Threads." Simplified architecture diagram from `ARCHITECTURE.md`.	45s

3	**LIVE DEMO**	`docker compose up` → ISS telemetry in the Yamcs web UI in 60 s. Swap TLE via curl → orbit updates within a second. Click PING in Yamcs → packet arrives in Spring Boot logs. **This is the strongest moment.**	90s
4	**Technical Depth**	Three bullet points: (a) CCSDS 133.0-B-1 at the ByteBuffer level, (b) TEME→ITRF→WGS84 via IERS-2010, (c) hot-swap TLE via `AtomicReference` with zero downtime. Goal: show that this is not an AI wrapper — I understand the physics and the protocols.	45s
5	**Differentiators / Killer Features**	Feature A (Predictive Orbital Shadowing) — the only true digital twin on the market; Feature B (Virtual Thread scaling) — 10,000+ sats per server; Feature C (Closed-loop command verification) — FinTech-grade reliability.	45s
6	**Commercialisation + Team**	Target customers (NewSpace, ESA BIC, small agencies, defence). Revenue model (open-core SaaS, per-sat licensing, ESA BIC runway). Scalability via student team (13 tickets, 2 juniors, 6 months).	45s
7	**Ask / Next Steps**	What I want from the programme: go-to-market mentoring, ESA BIC pipeline access, Advisory Board feedback on killer features, revenue model validation, recruiting contacts for junior team members.	30s

Total: ~5.5 minutes pitch + ~2 minutes live demo = safely within a 7-minute slot.

Three-Sentence Project Description (for the "short project description" field)

> **Palantir** is an open-source Ground Segment as a Service Digital Twin — a functional prototype that bridges Orekit 12.2 astrodynamics (SGP4/SDP4) with the Yamcs mission control system via the CCSDS 133.0-B-1 Space Packet protocol, launches with a single `docker compose up`, and today already delivers hot-swap TLE propagation via REST API, CCSDS telemetry downlink with XTCE decoding into Yamcs parameters, and a bidirectional TM/TC loop. The goal is to build a scalable cloud-native alternative to expensive legacy mission control systems for NewSpace operators of small-sat constellations — the 6-month Spaceport_SK roadmap targets three killer features: predictive orbital fault detection via model residual analysis, horizontal scaling for mega-constellations by combining Java 21 Virtual Threads (concurrent per-satellite I/O) with multi-core platform-thread parallelism (SGP4 CPU compute), and closed-loop command ACK verification through an XTCE `ManualVerifier` safety gate. The concrete Demo Day deliverable (29 September 2026) is an end-to-end Automated Collision Avoidance flow: PAL-501 HPC conjunction assessment against the CelesTrak debris catalogue, PAL-502 `FIRE_THRUSTER` commanding with mandatory operator approval, and a Phase 3b physics reaction where the operator sees a visible orbit change in the Yamcs UI after clicking APPROVE.

Team Composition

- **Jakub Toth** — solo founder and core engineer. Background in Java 21, Spring Boot, distributed systems, CCSDS, Orekit. The sprint plan for the Demo Day target (PAL-501 + PAL-502 + Phase 3b physics reaction) is **viable solo** with the descope strategy defined in Section 5; student involvement accelerates the timeline and deepens scope, but is not a blocker.
- **Planned expansion** — actively recruiting student contributors from **UMB Banská Bystrica** (project presented in early April 2026, awaiting commitment) for pair-programming on PAL-501 (Epic 5 — HPC Conjunction Assessment). The Demo target holds regardless of the UMB response.
- **Advisory** — seeking mentors from the space sector via the programme (ideally with Yamcs operational experience or ESA BIC relevance).

---

8. Risks & Open Questions (to Resolve BEFORE Applying)

Risks

1. **Could the jury object to "just" an open-source ground segment without a hardware/payload component?** Previous winners had tangible products (glider, AI model). Mitigation: strong live demo + a concrete commercial angle (GSaaS market sizing).
2. **The commercialisation path is not fully clarified.** Open-core SaaS is a valid model, but it must be developed much more concretely for Phase 1 (Business Canvas workshop). Early validation from 2–3 potential customers is needed BEFORE the programme.
3. **The student scale-up plan is ambitious.** Can I realistically onboard 2 students under these conditions? The UMB Banská Bystrica presentation happened in early April 2026 — awaiting their response. The solo fallback plan must remain viable (see Section 5, subsection *Concrete Demo Day 2026 Target*).
4. **Team = solo founder.** The programme does not block this, but mentors often push for "co-founder search." Prepare a clear answer for why solo is appropriate at this stage.
5. **August holiday gap.** Sprint 5 (August) falls into the traditional EU/SK holiday month. If UMB students join, part of their capacity will be unavailable; the user must also explicitly guard against losing the month personally. Mitigation: pull-in critical Sprint 5 tasks into July (Sprint 4); hard-deadline for Phase 3b wiring no later than 25 August.
6. **Sprint 6 (September) has no real buffer.** Demo Day is 29 September and Advisory Board feedback also lands in September. If Sprint 5 slips, September is not a "buffer phase" — it is the only integration + rehearsal window. Mitigation: a Critical Gate review on 1 September (end-to-end demo on a clean `docker compose up` from `master`); if the gate fails, activate the emergency descope plan.

Open Questions

- [ ] Verify the exact ESA BIC Slovakia conditions and figures (financial support, equity, deadlines).

- [ ] Verify whether the programme accepts individual founders with a scale-up plan (or whether a co-founder is required at application time).
- [ ] Secure 2–3 discovery calls with potential customers (a Slovak satellite startup? Needronix? SpaceScavengers? Datafrost?). This would produce validation material for Phase 1.
- [ ] Set up a public GitHub repository (if not already public) — without it, the open-source trust factor in the demo is weakened.
- [ ] Prepare a 60-second demo video (backup in case the live demo fails due to network issues on Demo Day).
- [ ] Confirm whether the application should be written in Slovak or English (the programme has international mentors — likely SK for the application, EN for the pitch deck).
- [ ] Follow up on the UMB response regarding student involvement in `FEATURE-v2-ext.md` tickets (priority: PAL-501 pair-programming).

---

9. Action Items — To Do Before 20 April 2026

Must-Have (Before the Deadline)

1. **Finalise the pitch draft** — one paragraph (application) + 7-slide deck (Demo Day ready-ish).
2. **Define the project theme** — proposed wording: "*Palantir: Open-Source GSaaS Digital Twin for NewSpace Mega-Constellations*".
3. **Write the short project description** — use the three-sentence version from Section 7.
4. **Team composition** — likely solo founder + expansion plan.
5. **Submit the application** to spaceoffice@sario.sk with a subject such as "*Spaceport_SK Application 2026 — Palantir Digital Twin*".

Nice-to-Have (Raises Odds, Not a Blocker)

6. **Public GitHub repository** — open-source messaging is more credible with visible code.
7. **README.md polish** — add a "Live Demo" section with a GIF or screenshot of the Yamcs UI.
8. **2–3 discovery calls** with real operators (provides material for the Phase 1 Business Canvas).
9. **CesiumJS proof of concept** (PAL-101 at least at a "Hello World" 3D globe level with a dummy position) — the visual hook for Demo Day is enormous.
10. **Follow-up on the UMB presentation** — track the response regarding student involvement; if it falls through, activate the solo descope strategy from Section 5.

Post Kick-Off (28 April Onward)

- Phase 1 (April-May): Business Canvas workshops, technology validation → prepare data for the GTM pitch in June.
- Phase 2 (June): Pitch deck polish, VC Q&A, ESA BIC conversation → secure a commitment on validation funding.

- June-September: Implement at least one of the killer features (Feature A = Predictive Orbital Shadowing is the strongest candidate, being the sharpest differentiator and technically feasible within 3 months).
- September: Demo Day rehearsal, final Advisory Board feedback, submission.

---

10. References

- **Spaceport_SK**: https://spaceoffice.sk/spaceport-sk/
- **Applications**: spaceoffice@sario.sk
- **Internal Palantir documents**:
  - `README.md` — project overview, build commands, REST API
  - `ARCHITECTURE.md` — full source dump, container config, XTCE MDB
  - `FLOW.md` — byte-level walkthrough of CCSDS encoding
  - `FEATURES.md` — implementation status, Phase 1–3 roadmap
  - `FEATURES_v2.md` — killer features A/B/C for the ESA BIC pitch
  - `FEATURE-v2-ext.md` — 13-ticket student integration backlog
- **External standards**:
  - CCSDS 133.0-B-1 (Space Packet Protocol) — https://public.ccsds.org/Pubs/133x0b2e1.pdf
  - Orekit — https://www.orekit.org/
  - Yamcs — https://yamcs.org/